SIMATS ENGINEERING

ASSESSMENT TOOL

**COURSE NAME: Theory of Computation** **COURSE CODE: CSA1304**

**COURSE OUTCOME COVERED**

**CO3: Construct regular expressions, and use the pumping lemma and closure properties to analyze regular languages. (BL:3)**

*Assessment Tool Weightage: 40%*

**QUESTIONS: 5** **Total Marks:100**
**Duration : 1 Hour**

# Experiments

***Code of Conduct:***

*I Yuvann Nithish R (192411267) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student: Yuvann Nithish R***

| Sl. No. | Lab Experiments - Questions |
|---|---|
| 1 | Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)<br>S → 0A1 A → 0A \| 1A \| ε |
| 2 | Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)<br>S → 0S0 \| 1S1 \| 0 \| 1 \| ε |
| 3 | Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)<br>S → 0S0 \| A A → 1A \| ε |
| 4 | Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)<br>S → 0S1 \| ε |
| 5 | Write a C program to check whether a given string belongs to the language defined by a Context Free Grammar (CFG)<br>S → A101A A → 0A \| 1A \| ε |

SIMATS ENGINEERING

ASSESSMENT TOOL

**SOLUTIONS**

## 1. CFG: S → 0A1, A → 0A | 1A | ε

### Language

This grammar generates strings that:

Start with 0

End with 1

Have any combination of 0 and 1 in between

Examples: 01, 001, 011, 0101, 001101

### C Program

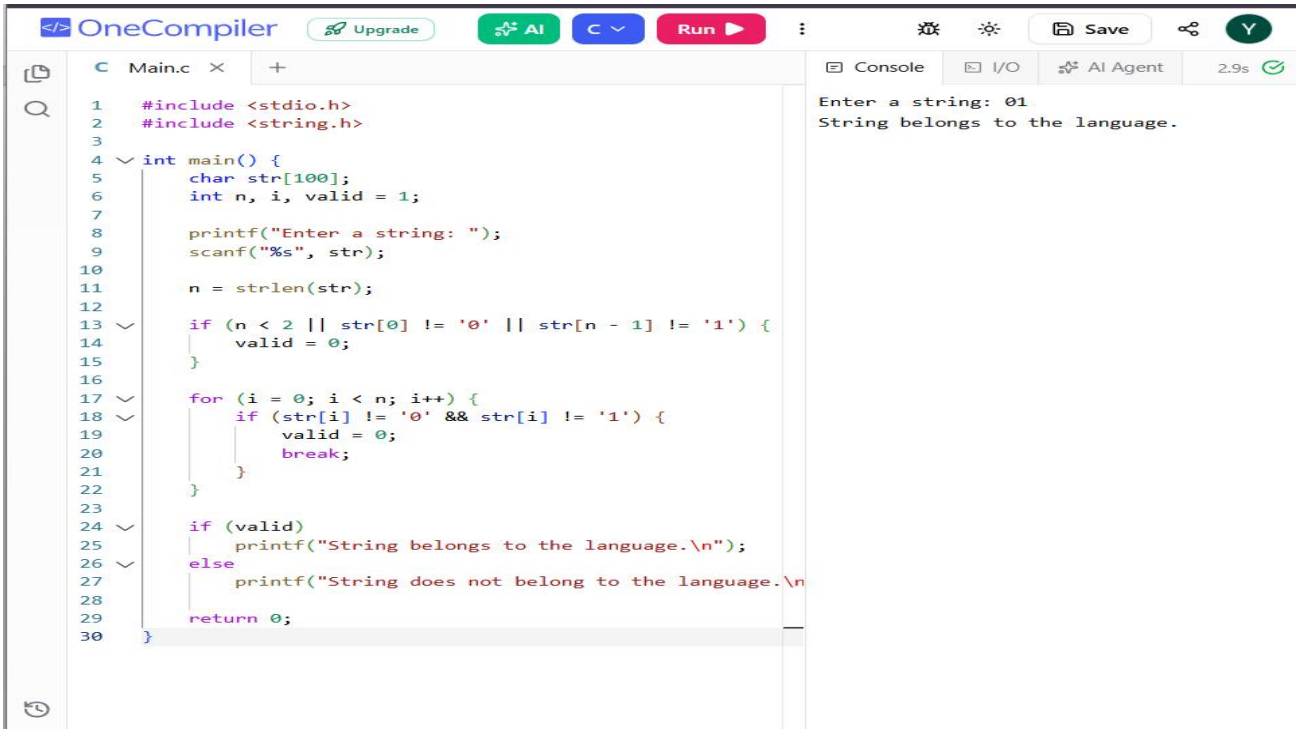
```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    int n, i, valid = 1;

    printf("Enter a string: ");
    scanf("%s", str);

    n = strlen(str);

    if (n < 2 || str[0] != '0' || str[n - 1] != '1') {
        valid = 0;
    }
```

```
    for (i = 0; i < n; i++) {
        if (str[i] != '0' && str[i] != '1') {
            valid = 0;
            break;
        }
    }

    if (valid)
        printf("String belongs to the language.\n");
    else
        printf("String does not belong to the language.\n");

    return 0;
}
```

**Example:**

Enter a string: 00101
String belongs to the language.

Output:



---

**2. CFG: S → 0S0 | 1S1 | 0 | 1 | ε**

**Language**

This grammar generates **palindromes** containing only 0 and 1.

Examples:

ε
0
1
00
11
010
101
0110
1001

**C Program**

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    int i, n, valid = 1;

    printf("Enter a string: ");
    scanf("%s", str);

    n = strlen(str);

    for (i = 0; i < n; i++) {
        if (str[i] != '0' && str[i] != '1') {
            valid = 0;
            break;
        }
    }

    if (valid) {
```

```
        for (i = 0; i < n / 2; i++) {
            if (str[i] != str[n - i - 1]) {
                valid = 0;
                break;
            }
        }
    }

    if (valid)
        printf("String belongs to the language.\n");
    else
        printf("String does not belong to the language.\n");

    return 0;
}
```

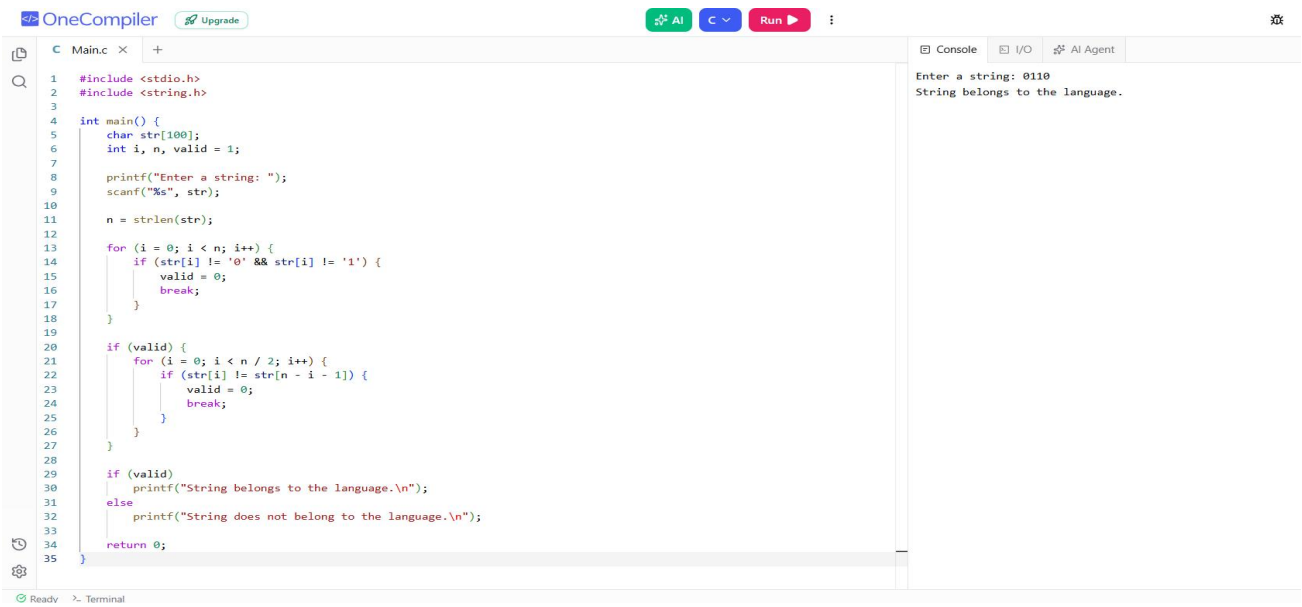
**Example:**

Enter a string: 0110
String belongs to the language.

Because:

0110
↑↑
Palindrome

output :



---

**3. CFG: S → 0S0 | A, A → 1A | ε**

**Language**

This generates strings of the form:

$0^n\ 1^m\ 0^n$

That means:

Same number of 0s on both sides

Any number of 1s in the middle

Examples:

ε
1
11
010
0110
00100
0011100

**C Program**

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    int n, left = 0, right, i;
    int valid = 1;

    printf("Enter a string: ");
    scanf("%s", str);

    n = strlen(str);
    right = n - 1;
```

```
    while (left < n && str[left] == '0')
        left++;

    while (right >= 0 && str[right] == '0')
        right--;

    /* Check that the middle part contains only 1s */
    for (i = left; i <= right; i++) {
        if (str[i] != '1') {
            valid = 0;
            break;
        }
    }

    /* Count zeros on both sides */
    if (valid) {
        int leftZeros = left;
        int rightZeros = n - 1 - right;

        if (leftZeros != rightZeros)
            valid = 0;
    }

    if (valid)
        printf("String belongs to the language.\n");
    else
        printf("String does not belong to the language.\n");

    return 0;
}
```

**Example:**

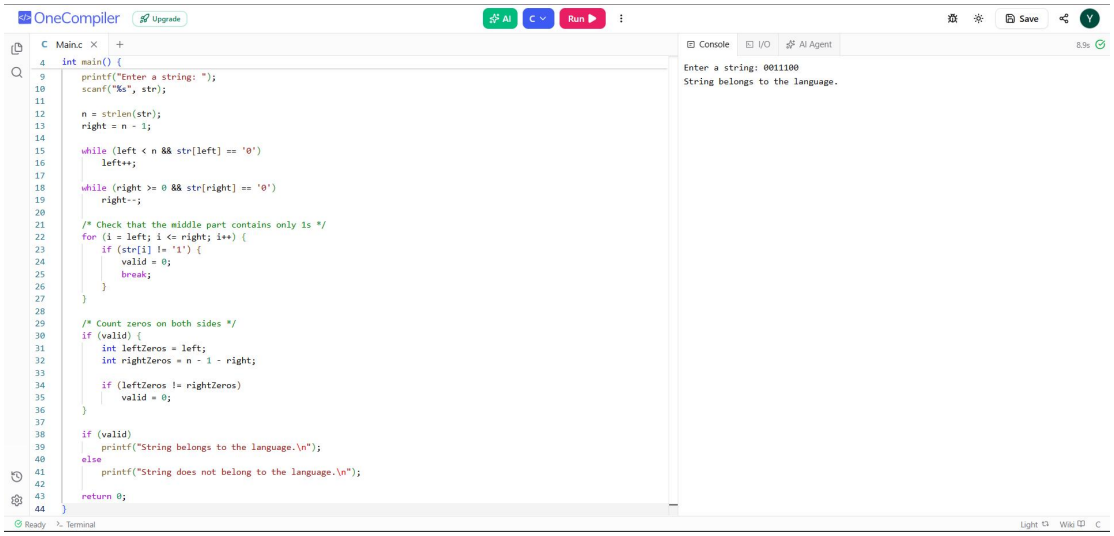
Enter a string: 0011100
String belongs to the language.

Because:

00 111 00

There are **2 zeros on both sides**.

## Output:




---

## 4. CFG: S → 0S1 | ε

## Language

This grammar generates:

$0^n1^n$

So the number of 0s must be exactly equal to the number of 1s, with all 0s before all 1s.

Examples:

ε
01
0011
000111
00001111

Invalid:

0
1
011
001
0101

**C Program**

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    int n, i, zeros = 0, ones = 0;
    int foundOne = 0;
    int valid = 1;

    printf("Enter a string: ");
    scanf("%s", str);

    n = strlen(str);

    for (i = 0; i < n; i++) {

        if (str[i] == '0') {
            if (foundOne) {
                valid = 0;
                break;
            }
            zeros++;
        }
        else if (str[i] == '1') {
            foundOne = 1;
            ones++;
        }
        else {
            valid = 0;
            break;
        }
    }
```

```
    if (zeros != ones)
        valid = 0;

    if (valid)
        printf("String belongs to the language.\n");
    else
        printf("String does not belong to the language.\n");

    return 0;
}
```

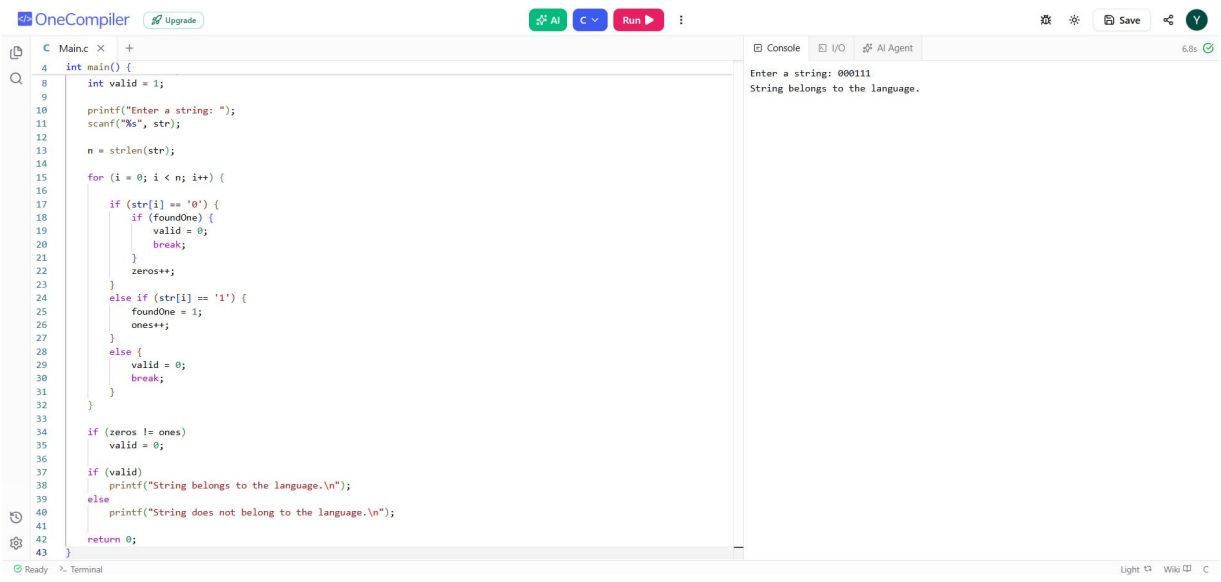
**Example:**

Enter a string: 000111
String belongs to the language.

Because:

000 111
 3   3

output:



**5. CFG: S → A101A, A → 0A | 1A | ε**

**Language**

Since A can generate **any binary string**, the grammar generates:

(binary string) 101 (binary string)

In simple terms:

> The string must contain 101 as a substring.

Examples:

101
0101
1010
11010
001011

**C Program**

```
#include <stdio.h>
#include <string.h>

int main() {
   char str[100];
   int n, i, found = 0;

   printf("Enter a string: ");
   scanf("%s", str);

   n = strlen(str);

   /* Check whether all characters are 0 or 1 */
   for (i = 0; i < n; i++) {
      if (str[i] != '0' && str[i] != '1') {
         printf("String does not belong to the language.\n");
         return 0;
      }
   }

   /* Search for substring 101 */
```

```
    for (i = 0; i <= n - 3; i++) {
        if (str[i] == '1' &&
            str[i + 1] == '0' &&
            str[i + 2] == '1') {

            found = 1;
            break;
        }
    }

    if (found)
        printf("String belongs to the language.\n");
    else
        printf("String does not belong to the language.\n");

    return 0;
}
```

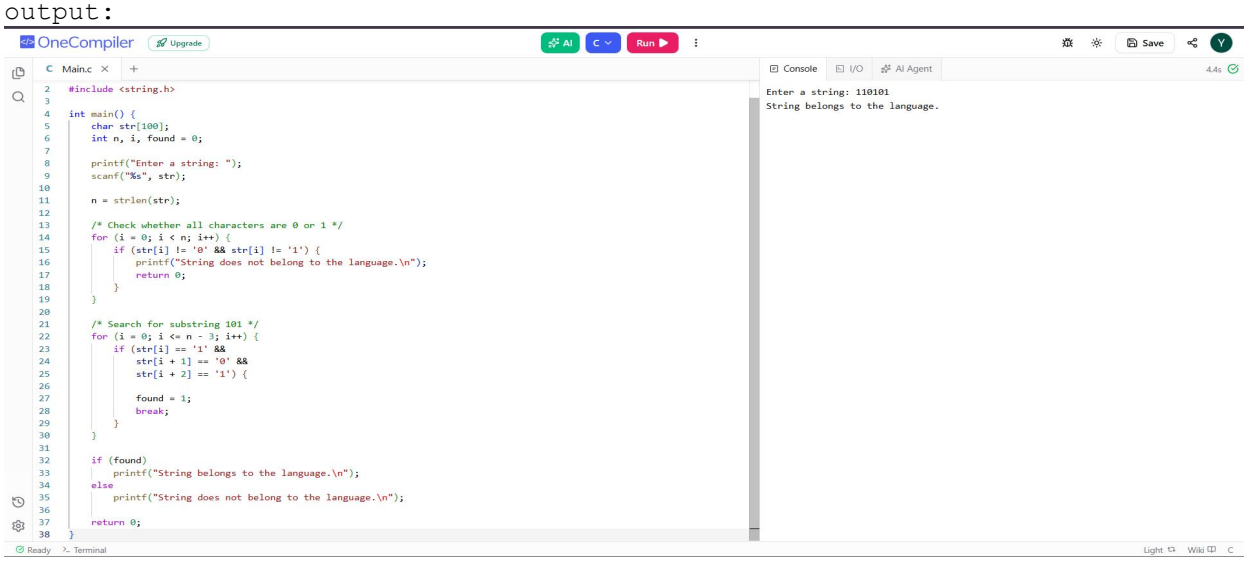
**Example:**

Enter a string: 110101
String belongs to the language.

Because it contains:

11 101 01
  ↑↑↑

output:



## RUBRICS FOR CALCULATION

| Criteria | Weightage | Level 4 – Exemplary (5 Marks) | Level 3 – Proficient (4 Marks) | Level 2 – Developing (2–3 Marks) | Level 1 – Beginning (0–1 Mark) |
|---|---|---|---|---|---|
| **Conceptual Understanding** | **30** | Demonstrates clear and complete understanding of the concept | Good understanding with minor gaps | Basic understanding with some misconceptions | Poor or incorrect understanding |
| **Accuracy of Answer** | **25** | Completely correct answer with no errors | Mostly correct with minor errors | Partially correct with noticeable errors | Incorrect or irrelevant answer |
| **Application of Knowledge** | **20** | Correctly applies concepts to solve the question/problem | Applies concepts with minor mistakes | Limited or partially correct application | No or incorrect application |
| **Completeness of Response** | **15** | Covers all required parts of the question | Covers most parts with minor omissions | Covers only some parts | Incomplete or missing response |
| **Clarity & Presentation** | **10** | Clear, concise, and well-organized answer | Understandable with minor issues | Somewhat unclear or poorly structured | Difficult to understand |